

CSC 4553

Fall 2026 - Day 11

Admin

- No lab this week, no quiz this week
- Exam 1 Thursday in-lecture
 - One page of notes, single-sided, *handwritten*

pthread

Questions to consider

- How do we actually create and manage threads in C?
- What happens when two threads touch the same variable?
- How do threads coordinate without busy-waiting?

POSIX threads (pthreads)

- Standard C threading API on Linux/macOS/BSD (`#include <pthread.h>`, link with `-lpthread`)
- Backed by 1:1 kernel threads on Linux (NPTL)
- Core primitives:
 - `pthread_create` / `pthread_join` - spawn and wait
 - `pthread_mutex_t` - mutual exclusion
 - `pthread_cond_t` - condition variables

Creating and joining

```
1  #include <pthread.h>
2  #include <stdio.h>
3
4  void *worker(void *arg) {
5      int id = *(int *)arg;
6      printf("hello from thread %d\n", id);
7      return NULL;
8  }
9
10 int main(void) {
11     pthread_t t1, t2;
12     int a = 1, b = 2;
13     pthread_create(&t1, NULL, worker, &a);
14     pthread_create(&t2, NULL, worker, &b);
15     pthread_join(t1, NULL);
16     pthread_join(t2, NULL);
17     return 0;
18 }
```

The problem: shared state

- Two threads, both running `counter++` one million times
- `counter++` is *not* atomic - it compiles to load, add, store
- Threads can interleave between those steps and clobber each other
- Result: final value is some number less than 2,000,000, and it's different every run
- This is a **race condition**: correctness depends on timing

Mutexes (preview)

- A **mutex** (mutual exclusion lock) enforces that only one thread at a time is inside a **critical section**
- `pthread_mutex_lock` blocks until the lock is available, then acquires it
- `pthread_mutex_unlock` releases it so another waiter can proceed
- We'll dig into *how* these work (hardware atomics, spinlocks, futexes) in the coming weeks

Mutex example

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2 long counter = 0;
3
4 void *bump(void *arg) {
5     for (int i = 0; i < 1000000; i++) {
6         pthread_mutex_lock(&lock);
7         counter++;           // critical section
8         pthread_mutex_unlock(&lock);
9     }
10    return NULL;
11 }
```

- Now the increment is safe - but threads may spend time waiting
- Keep critical sections **small**: lock, touch shared data, unlock

Condition variables (preview)

- Mutexes answer “who gets in?” - condition variables answer “when should I wake up?”
- Use case: consumer thread has nothing to do until the producer puts something in the queue
- Alternative - busy-waiting (`while (queue_empty()) {}`) - burns CPU for nothing
- A **condition variable** lets a thread sleep until another thread signals that a condition may now be true

Condvar mechanics

- Always paired with a mutex that protects the shared state being checked
- `pthread_cond_wait(&cv, &lock)` atomically:
 - releases `lock`
 - puts the thread to sleep
 - re-acquires `lock` when woken
- `pthread_cond_signal(&cv)` wakes one waiter; `pthread_cond_broadcast` wakes all
- Always wait in a `while loop`, not `if` - spurious wakeups happen, and another thread may have consumed the condition first

Producer/consumer sketch

```
1 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
3 queue_t q;
4
5 void *consumer(void *) {
6     pthread_mutex_lock(&m);
7     while (queue_empty(&q))
8         pthread_cond_wait(&not_empty, &m);
9     item_t x = queue_pop(&q);
10    pthread_mutex_unlock(&m);
11    process(x);
12    return NULL;
13 }
14
15 void *producer(void *) {
16     item_t x = make_item();
17     pthread_mutex_lock(&m);
18     queue_push(&q, x);
19     pthread_cond_signal(&not_empty);
```

Takeaways

- pthreads gives you a thin, portable interface to 1:1 kernel threads
- Shared memory is the blessing *and* the curse of threading
- Mutexes guard data; condition variables coordinate *events*
- After the exam we'll talk about what's happening underneath these calls, and on the ways concurrent code can still go wrong (deadlock, starvation, priority inversion, ...)

What isn't clear?

Comments? Thoughts?